شتاب‌دهنده‌های سخت‌افزاری

دانشکده مهندسی کامپیوتر

تابستان ۱۴۰۴

استاد درس: دارا رحمتی

طراحی پروژه: زهرا تاکی و متین فیروزبخت

پروژه پایانی

تاریخ انتشار : ۱۴۰۴/۰۴/۰۶ - تاریخ تحویل فاز اول : ۱۴۰۴/۰۴/۱۶ - تاریخ تحویل فاز دوم : ۱۴۰۴/۰۵/۰۶

در این پروژه، هدف پیاده‌سازی یک شتابدهنده سخت‌افزاری[۱] LeNet-۵ با استفاده از آرایه‌های سیستولیک[۲] با جریان داده‌ای[۳] خروجی‌ایستا[۴] و بهینه‌سازی مدل نرم‌افزاری است.

۱. مقدمه

شبکه‌های عصبی کانولوشنی[۵] موفقیت چشمگیری در حوزه‌های مختلف، به‌ویژه در وظایف پردازش تصویر[۶] و بینایی کامپیوتر[۷] مانند طبقه‌بندی[۸] و تشخیص اشیاء[۹]، از خود نشان داده‌اند. با این حال، نیازهای محاسباتی و حافظه‌ای قابل توجه شبکه‌های عصبی کانولوشنی مدرن، اغلب مانع از استقرار آن‌ها بر روی دستگاه‌های با منابع محدود یا در کاربردهایی می‌شود که نیازمند عملکرد بی‌درنگ[۱۰] هستند. این چالش، تحقیقات و توسعه گسترده‌ای را در زمینه شتاب‌دهنده‌های سخت‌افزاری اختصاصی و تکنیک‌های بهینه‌سازی مدل به راه انداخته است.

این پروژه، برای توسعه و ارزیابی یک شتاب‌دهنده سخت‌افزاری برای شبکه LeNet-۵، که یک معماری CNN بنیادی است طراحی شده‌است. در تمرین سوم درس دیدیم که شبکه ابتدا بر روی مجموعه داده[۱۱] Fashion MNIST آموزش داده شد و سپس پیاده‌سازی سخت‌افزاری آن با استفاده از آرایه‌های سیستولیک و با جریان‌داده‌های مختلف به انجام رسید. در این پروژه، در ادامه‌ی تمرین سوم، در دو فاز نرم‌افزاری و سخت‌افزاری سعی در استفاده و بررسی روش‌های بهینه‌سازی داریم.

در فاز نرم‌افزاری، در کنار آموزش مدل پایه، ما بهینه‌سازی مدل را از طریق هرس شبکه (هرس وزن و هرس نورون/فیلتر) و کوانتیزاسیون (پس از آموزش) به اعداد صحیح ۸ بیتی بررسی کردیم تا تأثیرات آن‌ها را بر دقت[۱۲] و قابلیت فشرده‌سازی[۱۳] مدل درک

---

۱. Hardware Accelerator
۲. Systolic Array
۳. Dataflow
۴. Output Stationary
۵. Convolution Neural Network (CNN)
۶. Image Processing
۷. Computer Vision
۸. Classification
۹. Object Detection
۱۰. Real-time Action
۱۱. Dataset
۱۲. Accuracy
۱۳. Data compression

کنیم. سپس در فاز سخت‌افزاری یک شتاب‌دهنده سخت‌افزاری برای مدل پایه LeNet-۵ را بر اساس معماری آرایه سیستولیک، که یک الگو[۱۴] پردازش موازی[۱۵] بسیار مناسب برای ضرب‌های ماتریس-بردار[۱۶] ذاتی در CNN ها است، طراحی و پیاده‌سازی کردیم.

طراحی سخت‌افزاری پروژه با استفاده از یک زبان توصیف سخت‌افزار پیاده‌سازی[۱۷] و برای یک FPGA با استفاده از مجموعه طراحی Xilinx Vivado سنتز شدند. ارزیابی شامل مقایسه سرعت شتاب‌دهنده‌های سخت‌افزاری در برابر یک پیاده‌سازی نرم‌افزاری بود. علاوه بر این، نوع جریان داده سخت‌افزاری از نظر مصرف توان روی تراشه[۱۸]، توان پردازشی[۱۹] و بهره‌برداری از منابع FPGA با یکدیگر مقایسه شدند. تأثیر پیاده‌سازی سخت‌افزاری بر دقت طبقه‌بندی نیز ارزیابی شد.

این پروژه به شرح زیر سازماندهی شده است:

در بخش ۲، پیش‌زمینه‌ای در مورد شبکه LeNet-۵ ، مجموعه داده Fashion MNIST ، استراتژی‌های جریان داده و آرایه‌های سیستولیک ارائه می‌دهد. بخش ۳، جزئیات آموزش نرم‌افزاری، هرس مدل[۲۰] و کوانتیزاسیون پس از آموزش[۲۱] LeNet-۵ را شرح می‌دهد. بخش ۴، معماری سخت‌افزار و پیاده‌سازی جریان‌های داده مختلف برای مدل پایه را توصیف می‌کند. بخش ۵، تنظیمات آزمایش و نتایج ارزیابی را ارائه می‌دهد.

۲. پیش‌زمینه

الف. معماری LeNet-۵

معماری LeNet-۵ ،که توسط LeCun و همکارانش پیشنهاد شد، یک CNN اولیه و تأثیرگذار است. این معماری معمولاً از چندین لایه تشکیل شده است، از جمله لایه‌های کانولوشنی برای استخراج ویژگی، لایه‌های تجمعی مانند ادغام[۲۲] و زیرنمونه‌گیری[۲۳] برای کاهش ابعاد، و لایه‌های تماماً متصل[۲۴] برای طبقه‌بندی. برای بخش شتاب‌دهی سخت‌افزاری این پروژه، ورودی LeNet-۵ یک تصویر ۳۲x۳۲ است. پیکربندی مشخص در جدول ۱ شرح داده شده است. برای فاز اول پروژه ، بهینه‌سازی نرم‌افزاری که هرس مدل و کوانتیزاسیون پس از آموزش است. از یک پیکربندی کمی متفاوت از LeNet-۵ که برای ورودی‌های ۲۸x۲۸ که اندازه اصلی Fashion MNIST تطبیق داده شده بود، استفاده شد که جزئیات آن در بخش ۳ آمده است.

---

[۱۴] .Paradigm
[۱۵] .Parallel Processing
[۱۶] .Matrix-Vector Multiplication
[۱۷] .Hardware description language (HDL)
[۱۸] . On-chip power consumption
[۱۹] .Throughput
[۲۰] .Model Pruning
[۲۱] .Post training Quantization
[۲۲] .Pooling
[۲۳] .Subsampling
[۲۴] .Fully-connected layers

جدول ۱: معماری LeNet-5 برای شتاب‌دهی سخت‌افزاری (ورودی ۳۲x۳۲)

Layer	Type	Input Size	Kernel	Stride	Output Size
Input	-	32x32x1	-	-	28x28x1
Conv1	Conv	32x32x1	5x5, 6 filters	1	28x28x6
S2	Pool	28x28x6	2x2	2	14x14x6
Conv3	Conv	14x14x6	5x5, 16 filters	1	10x10x16
S4	Pool	10x10x16	2x2	2	5x5x16
Conv5	Conv	5x5x16	5x5, 120 filters	1	1x1x120
F6	FC	120 units	-	-	84 units
Output	FC	84 units	-	-	10 units

ب. مجموعه داده Fashion MNIST

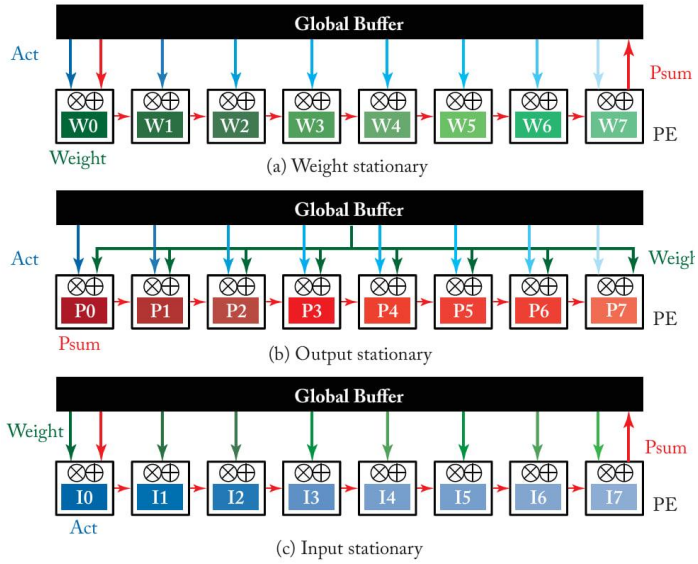
مجموعه داده Fashion MNIST برای آموزش و ارزیابی مدل LeNet-۵ استفاده شد. این مجموعه شامل ۷۰,۰۰۰ تصویر سیاه و سفید از اقلام پوشاک است (۶۰,۰۰۰ برای آموزش، ۱۰,۰۰۰ برای تست). هر تصویر ۲۸x۲۸ پیکسل است. برای بخش شتاب‌دهنده سخت‌افزاری که به ورودی ۳۲x۳۲ نیاز داشت، تصاویر با صفر لایه‌گذاری[۲۵] شدند. برای مطالعات بهینه‌سازی نرم‌افزاری، از تصاویر اصلی ۲۸x۲۸ استفاده شد.

ج. استراتژی‌های جریان داده در شتاب‌دهنده‌های CNN

استراتژی‌های جریان داده نحوه سازماندهی و جابجایی داده‌ها مانند وزن‌ها[۲۶]، فعال‌سازی‌های ورودی[۲۷]، مجموع‌های جزئی[۲۸] را در عناصر پردازشی یک شتاب‌دهنده سخت‌افزاری تعیین می‌کنند و تأثیر قابل توجهی بر عملکرد و کارایی دارند .سه استراتژی رایج که عموما مورد استفاده قرار می‌گیرند، در شکل ۱ نیز نمایش داده شده‌اند:

- وزن‌ایستا[۲۹] : وزن‌ها در عناصر پردازشی ثابت می‌مانند.
- خروجی‌ایستا[۳۰] : مجموع‌های جزئی برای خروجی‌ها در عناصر پردازشی ثابت می‌مانند.
- ورودی‌ایستا[۳۱] : فعال‌سازی‌های ورودی در عناصر پردازشی ثابت می‌مانند.

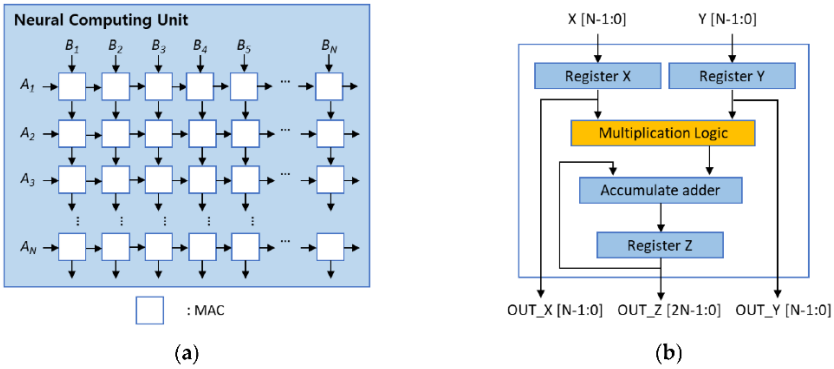
[۲۵] .Zero-padding
[۲۶] .Weights
[۲۷] .Input Activations
[۲۸] .Partial Sums
[۲۹] .Weight-Stationary (WS)
[۳۰] .Output-Stationary (OS)
[۳۱] . Input-Stationary (IS)



شکل ۱ - نمایش سه استراتژی جریان داده در معماری شتاب‌دهنده: (a) وزن‌ایستا، (b) خروجی‌ایستا، و (c) ورودی‌ایستا.

د. معماری آرایه سیستولیک

آرایه‌های سیستولیک معماری‌های موازی هستند که برای وظایف محاسباتی سنگین مانند ضرب ماتریس درCNN ها ایده‌آل می‌باشند. یک آرایه سیستولیک دو-بعدی از عناصر پردازشی[۳۲] طراحی شد که هر PE یک عملیات ضرب-انباشت انجام می‌دهد. شکل۲ نمونه‌ای از یک آرایه سیستولیک دو-بعدی و ساختار داخلی عنصر پردازشی آن را نشان می‌دهد.



شکل ۲- (a)ساختار یک آرایه سیستولیک دو-بعدی شامل واحدهای MAC برای ضرب و جمع ماتریس. (b) معماری داخلی یک عنصر پردازشی (PE) با رجیستر ورودی، منطق ضرب، جمع‌کننده و رجیستر انباشتی.

[۳۲] . Processing Element (PE)

۳. پیاده‌سازی نرم‌افزاری، آموزش و بهینه‌سازی

مدل LeNet-۵ با استفاده از فریمورک‌های مختلف مانند پای‌تورچ[۳۳] و تنسورفلو[۳۴] آن در نرم‌افزار پیاده‌سازی و آموزش داده شد. از مجموعه دادهFashion MNIST استفاده گردید. برای مدل پایه‌ای که برای شتاب‌دهی سخت‌افزاری در نظر گرفته شده بود، تصاویر ۲۸x۲۸ با صفر به ابعاد ۳۲x۳۲ لایه‌گذاری شدند. برای مطالعات بهینه‌سازی نرم‌افزاری(هرس مدل و کوانتیزاسیون پس از آموزش LeNet-۵)، از تصاویر اصلی ۲۸x۲۸ با معماری LeNet-۵ که در جدول ۱ توضیح داده شده، استفاده شد.

الف. هرس شبکه LeNet-۵

در فرآیند بهینه‌سازی مدل LeNet-۵ با ورودی ۲۸×۲۸، از تکنیک‌های هرس به منظور کاهش تعداد پارامترها و در نتیجه کوچک‌سازی مدل و تسریع در مرحله استنتاج[۳۵] استفاده شد، بدون آن‌که تأثیر چشم‌گیری بر دقت مدل وارد شود. در این راستا، دو روش هرس مبتنی بر قدر مطلق پیاده‌سازی گردید.

۱. هرس وزن[۳۶]

در روش نخست که به صورت غیرساختاری انجام شد، وزن‌هایی که دارای کوچک‌ترین مقدار مطلق بودند، صفر گردیدند. این نوع هرس مستقیماً تعداد پارامترها را کاهش می‌دهد، بدون آن‌که تغییری در ساختار لایه‌ها ایجاد کند.

۲. هرس ساختاری[۳۷] (نورون/فیلتر)

در روش دوم که به‌صورت ساختاری اعمال شد، هرس در سطح نورون‌ها و فیلترها انجام گرفت. در لایه‌های تمام‌متصل، نورون‌هایی که مجموع مربعات وزن‌های ورودی آن‌ها (موسوم به نُرمL۲) کمترین مقدار را داشتند، حذف شدند. همچنین در لایه‌های کانولوشنی، فیلترهایی که وزن‌های آن‌ها دارای کمترین نرم L۲ بودند، به‌طور کامل از مدل حذف گردیدند.

لازم به ذکر است که نرم L۲ برای یک بردار وزنی، $w = w_۱, w_۲, \dots, w_n$ برابر است با:

$$\| w \|_۲ = \sqrt{\sum_{i=۱}^{n} w_i^۲}$$

که معیاری از اندازه کلی وزن‌ها است و در این پروژه برای شناسایی نورون‌ها و فیلترهای کم‌اثر مورد استفاده قرار گرفته است.استفاده از این معیار کمک می‌کند تا نورون‌ها یا فیلترهایی که سهم کمتری در خروجی مدل دارند، شناسایی و حذف شوند. این اقدامات با هدف افزایش کارایی و کاهش پیچیدگی مدل انجام شد، در حالی که تلاش گردید دقت مدل تا حد ممکن حفظ شود.

---

۳۳ . Pytorch
۳٤ . TensorFlow
۳۵ .Inference Step
۳٦ .Weight Pruning
۳۷ .Structure Pruning

ب. کوانتیزاسیون پس از آموزش شبکه LeNet-۵

علاوه بر هرس، کوانتیزاسیون پس از آموزش به عنوان یکی دیگر از تکنیک‌های بهینه‌سازی سطح نرم‌افزار مورد بررسی قرار گرفت تا بتوان اندازه مدل را کاهش داده و امکان استنتاج سریع‌تر را بر روی سخت‌افزارهایی که از محاسبات اعداد صحیح پشتیبانی می‌کنند، فراهم نمود. در این روش، وزن‌ها و فعال‌سازی‌های مدل ممیز شناور که از پیش آموزش دیده است، به اعداد صحیح با تعداد بیت کمتر، معمولاً ۸ بیت، تبدیل می‌شوند، بدون آنکه نیاز به آموزش مجدد مدل باشد.

این فرآیند بر روی مدل پایه LeNet-۵ با ورودی ۲۸×۲۸ و دقت ممیز شناور ۳۲ بیتی پس از آموزش روی مجموعه داده Fashion MNIST اعمال شد. برای پیاده‌سازی این تبدیل از ابزار (Pytorch/TensorFlow) استفاده گردید. یکی از مراحل کلیدی در کوانتیزاسیون پس از آموزش، مرحله کالیبراسیون است؛ در این مرحله، زیرمجموعه‌ای کوچک و نماینده از داده‌های آموزشی(معمولاً ۱۰۰ تا ۵۰۰ تصویر از مجموعه داده) از مدل عبور داده می‌شود تا اطلاعات آماری مربوط به دامنه فعال‌سازی‌ها جمع‌آوری شود. این آمار سپس برای تعیین پارامترهای بهینه کوانتیزاسیون، شامل مقیاس و نقطه صفر، به منظور تبدیل مقادیر فعال‌سازی ممیز شناور به اعداد صحیح INT۸ به کار می‌رود. وزن‌های مدل نیز معمولاً به صورت متقارن یا نامتقارن به INT۸ کوانتیزه می‌شوند.

اهداف اصلی اعمال کوانتیزاسیون INT۸ پس از آموزش عبارت بودند از کاهش چشمگیر اندازه مدل، چرا که پارامترهای INT۸به نسبت پارامترهای ممیز شناور ۳۲ بیتی حدود چهار برابر فضای کمتری اشغال می‌کنند، و فراهم کردن امکان استنتاج سریع‌تر بر روی پلتفرم‌های سخت‌افزاری مجهز به واحدهای پردازش اختصاصیINT۸ ، که به دلیل محاسبات ساده‌تر و کاهش نیاز به پهنای باند حافظه، کارایی بهتری ارائه می‌دهند. برای ارزیابی اثرات کوانتیزاسیون بر عملکرد مدل، دقت مدل کوانتیزه شده INT۸روی مجموعه تست Fashion MNIST سنجیده شد. نتایج کلیدی این ارزیابی در جدول ۲ ارائه شده است.

جدول ۲ : تأثیر کوانتیزاسیون پس از آموزش INT8 بر LENET-5

Model Version	Accuracy (%)	Model Size (MB)
Baseline FP32	90.53%	0.24 MB
INT8 Quantized	90.17%	0.06 MB

۳. پیاده‌سازی سخت‌افزاری

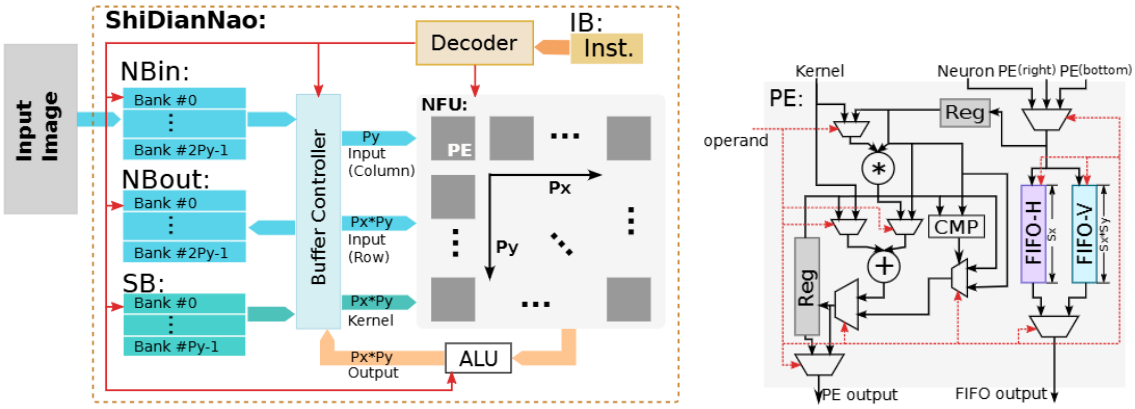
طراحی شتاب‌دهنده سخت‌افزاری با تمرکز روی مرحله استنتاج مدل LeNet-۵ (با استفاده از وزن‌های مدل بدون هرس و ورودی ۳۲×۳۲) انجام شد. پیاده‌سازی سخت‌افزاری استراتژی جریان داده WS، از بین جریان‌داده‌های معرفی شده، بر روی یک آرایه سیستولیک ۵×۵ از عناصر پردازشی به کار می‌گیرند.

الف. معماری آرایه سیستولیک

ساختار اصلی شتاب‌دهنده، یک آرایه سیستولیک دوبعدی از عناصر پردازشی است که هر PE عمل ضرب-جمع بین ورودی و وزن را انجام داده و نتیجه را به عنصر مجاور منتقل می‌کند.

ب. پیاده‌سازی جریان‌داده‌ها

جریان داده‌ها بر اساس استراتژی وزن‌ایستا تنظیم شد، به‌گونه‌ای که مجموع‌های جزئی در هر PE ثابت باقی می‌مانند و به تدریج به‌روزرسانی می‌شوند. در این پروژه میخواهیم **استراتژی خروجی‌ایستا** را پیاده‌سازی و بررسی کنیم. یکی از شابدهنده‌های معروف معرفی شده در کتاب ShiDianNao نام دارد. این شتابدهنده از جریان‌داده خروجی‌ایستا در یک شبکه NFU که با ساختار آرایه سیستولیک پیاده‌سازی شده است استفاده میکند. در شکل ۳ میتوانید معماری شتابدهنده و معماری PE را مشاهده کنید.



شکل ۳ – معماری شتاب‌دهنده ShiDianNao و ساختار PE معماری.

پ. معماری حافظه و سازمان‌دهی منابع داخلی

برای نگهداری وزن‌های آموزش‌دیده، فعال‌سازی‌ها و مجموع‌های جزئی، از ماژول‌های حافظه داخلی روی تراشه استفاده شد که با ساختار آرایه سیستولیک یکپارچه‌سازی شده‌اند. وزن‌های مدل LeNet-۵ پس از آموزش، به قالب ۸ بیتی با دقت INT۸ تبدیل و در حافظه‌ای با ظرفیت ۶۴ کیلوبایت ذخیره شدند تا امکان نگهداری حدود ۶۲ هزار وزن ۸ بیتی فراهم شود. این حافظه به صورت فقط–خواندنی طراحی شد و به‌گونه‌ای سازمان‌دهی شد که هر PE در آرایه سیستولیک بتواند به‌صورت مستقیم به وزن‌های مربوط به خود دسترسی داشته باشد. برای نگهداری ورودی‌ها، نقشه‌های ویژگی[۳۸] میانی و مجموع‌های جزئی، حافظه‌ای با ظرفیت ۱۶ کیلوبایت مورد استفاده قرار گرفت. این حافظه به صورت قابل‌خواندن و نوشتن طراحی شد.

ج. سلسله‌مراتب حافظه دو سطحی در پیاده‌سازی OS

برای بهبود کارایی و کاهش وابستگی به حافظه خارج از تراشه[۳۹]، در پیاده‌سازی با جریان داده خروجی‌ایستا یک ساختار حافظه با دو سطح را مورد بررسی قرار میدهیم.

سطح اول به صورت یک SRAM با ظرفیت ۱ کیلوبایت طراحی میشود که به‌صورت مستقیم نگاشته شده و دارای زمان دستیابی یک سیکل کلاک می‌باشد. باید در طراحی اندازه هر خط حافظه[۴۰] مشخص و در نظر گرفته شود تا توازن مناسبی بین

---

[۳۸] .Feature map
[۳۹] .Off-chip memory
[۴۰] .Cache line

بهره‌وری از حافظه و پیچیدگی مدیریت آن فراهم شود. این حافظه عمدتاً برای نگهداری موقت فعال‌سازی‌ها استفاده گردید تا داده‌های پرکاربرد به سرعت در دسترس باشند.

سطح دوم حافظه شامل حافظه‌های بلوکی[۴۱] روی تراشه با ظرفیت بیشتر و زمان دستیابی معادل ۴ سیکل کلاک میباشد. این حافظه نقش حافظه اصلی داخلی را ایفا می‌کرد و در آن وزن‌ها و فعال‌سازی‌ها به صورت بلوکی نگهداری می‌شدند. برای شبیه‌سازی عملکرد L۲ از یک ماشین حالت محدود یا یک شمارنده‌های اختصاصی برای مدیریت وضعیت انتقال داده و تشخیص خطای کش[۴۲] استفاده شود.

برای مدیریت حافظه بین این دو سطح، داده‌ها به‌صورت تقاضامحور[۴۳] از L۲ به L۱ منتقل میشوند. برای فعال‌سازی‌ها لازم است تا از سیاست‌های جایگزینی استفاده شود. به عنوان مثال سیاست جایگزینی اخیراً کمتر استفاده شده[۴۴] استفاده شود تا حافظه L۱ همیشه حاوی پرکاربردترین داده‌ها باشد.

استفاده از سلسله‌مراتب دو سطحی از نظر تاثیر بر کارایی باید بتواند منجر به کاهش تعداد دسترسی‌ها به حافظه خارج از تراشه شود. این امر باعث کاهش مصرف توان و بهبود زمان استنتاج[۴۵] میشود. پس از طراحی باید نتایج تاثیر بر کارایی بررسی شود.

د. پشتیبانی از کوانتیزاسیون

در طراحی سخت‌افزاری، پشتیبانی از کوانتیزاسیون به اعداد ۸ بیتی نیز لحاظ شد تا با مدل بهینه‌سازی‌شده نرم‌افزاری هم‌خوانی داشته باشد. به همین منظور، مسیر داده و ضرب‌کننده‌ها به‌گونه‌ای طراحی شدند که توانایی پردازش INT۸ را داشته باشند، و بخش انباشت نیز دارای دقت کافی (مثلاً INT۳۲) نظر گرفته شد.

ه. پشتیبانی از پراکندگی غیرساختاری

پراکندگی غیرساختاری[۴۶] به حالتی گفته می‌شود که مقادیر صفر به‌صورت نامنظم و پراکنده، بدون هیچ الگوی مشخص یا ساختار ثابتی، در سراسر ماتریس وزن یا نقشه ویژگی قرار دارند. در این نوع پراکندگی، هر وزن یا فعال‌سازی به‌صورت مستقل می‌تواند صفر یا غیرصفر باشد. در صورت انجام ندادن عملیات با عملوند صفر و پشتیبانی از پراکندگی میتوانیم مزایایی از جمله کاهش قابل توجه در تعداد عملیات ضرب-جمع و کاهش مصرف حافظه و توان داشته باشیم. از چالش‌های آن نیز میتوان به زمان‌بندی و هماهنگی پیچیده‌تر جریان‌داده اشاره کرد.

با توجه به استفاده از تکنیک هرس کردن در فاز نرم‌افزاری و در نتیجه پراکندگی در وزن‌ها یا فعال‌سازی‌ها، برای پشتیبانی از پراکندگی، طراحی عناصر پردازشی باید به‌گونه‌ای انجام شود که قابلیت پردازش داده‌های پراکنده را داشته باشند. در این طراحی، مکانیزم‌هایی برای صرف‌نظر کردن از محاسبات ضرب مربوط به مقادیر صفر در نظر گرفته میشود تا از انجام عملیات بی‌اثر جلوگیری کنیم.

---

[۴۱] .Block RAM (BRAM)
[۴۲] .Cache miss
[۴۳] . On-demand
[۴۴] . Least Recently Used (LRU)
[۴۵] . Inference latency
[۴۶] . Unstructured Sparsity

به منظور پشتیبانی از پراکندگی غیرساختاری، هر PE به یک واحد بررسی‌کننده‌ی صفر[۴۷]مجهز میشود که پیش از انجام ضرب، مقدار ورودی یا وزن را بررسی کرده و در صورت صفر بودن، مسیر داده را به‌صورت بای‌پس[۴۸] منتقل میکند. این کار موجب کاهش تعداد عملیات MAC واقعی، کاهش مصرف توان، و بهینه‌سازی بهره‌برداری از منابع منطقی FPGA میشود.

همانطور که اشاره شد یکی از چالش های پشتیانی از پراکندگی غیر ساختاری در زمان‌بندی و هماهنگی جریان داده است. برای فعال‌سازی کامل این ویژگی، باید جریان داده به‌گونه‌ای طراحی شود که هر PE بداند هر حاصل‌جمع جرئی تولید شده مربوط به کدام موقعیت است. به طور کلی یکی از راه‌های انتقال موقعیت مقادیر غیرصفر[۴۹] نیز به همراه داده‌ها است. در این پروژه باید روشی متناسب با جریان‌داده خروجی‌ایستا انتخاب کنید.

ی. سنتز و ارزیابی

پس از طراحی و پیاده‌سازی، پروژه سنتز شد و گزارش‌های حاصل از سنتز شامل مصرف منابع (LUT، FF، BRAM، DSP)، زمان اجرا (تاخیر مسیر بحرانی)، و مصرف توان به‌دست آمد.

٤. تنظیمات آزمایش و نتایج ارزیابی

الف. **فاز اول** : پیاده‌سازی نرم‌افزاری، آموزش و بهینه‌سازی

گام اول

- هرس کردن

در این مرحله، هدف کاهش تدریجی تعداد پارامترهای مدل از طریق اعمال هرس با نسبت‌های مختلف پراکندگی شامل: ۲۵٪، ۵۰٪، ۷۰٪، ۹۰٪، ۹۵٪، ۹۷٪ و ۹۹٪ می‌باشد. مراحل انجام کار به شرح زیر است:

۱. دو روش هرس (ساختاری و غیرساختاری) بر روی مدل اعمال کنید و نسبت‌های مختلف پراکندگی برای هر روش بررسی کنید.
۲. اندازه مدل (تعداد کل پارامترها) در حالت اولیه و پس از اعمال هر سطح از هرس محاسبه کنید.
۳. دقت مدل پیش از هرس و پس از اعمال هرس با نسبت‌های مختلف اندازه‌گیری و مقایسه کنید.

گام دوم

- کوانتیزاسیون پس از آموزش

در این بخش، به منظور کاهش اندازه مدل و آماده‌سازی آن برای اجرا بر روی سخت‌افزارهای مبتنی بر محاسبات عدد صحیح، فرآیند کوانتیزاسیون پس از آموزش بر روی مدل ممیز شناور اعمال می‌گردد:

---

[۴۷] . Zero-check unit
[۴۸] . Bypass
[۴۹] . Non-zero indices

۱. بدون نیاز به بازآموزی مدل وزن‌ها و فعال‌سازی‌های مدل ۳۲ بیتی ممیز شناور به مقادیر صحیح ۸ بیتی تبدیل می‌شوند.
۲. اندازه مدل در حالت اولیه ممیزشناور ۳۲بیتی و پس از کوانتیزاسیون عددصحیح ۸بیتی محاسبه و مقایسه کنید.
۳. دقت مدل قبل و بعد از کوانتیزاسیون با استفاده از مجموعه تست مورد ارزیابی قرار دهید.

گام سوم

- ترکیب هرس و کوانتیزاسیون

در این گام، ابتدا عملیات هرس بر روی مدل اعمال شده و سپس نسخه هرس‌شده تحت کوانتیزاسیون قرار می‌گیرد. هدف این مرحله بررسی اثربخشی ترکیبی دو تکنیک بهینه‌سازی است:

۱. اندازه مدل و دقت آن در حالت نهایی (پس از هرس و کوانتیزاسیون) محاسبه شده و با حالت‌های قبلی مقایسه می‌شود.

ب. **فاز دوم** : پیاده‌سازی سخت‌افزاری

- پیاده‌سازی استنتاج شبکه LeNet-۵ با جریان داده خروجی‌ایستا و یک سطح حافظه

در این گام، بر روی یک شبکه آرایه سیستولیک و جریان‌داده خروجی‌ایستا، پنج لایه‌ی شبکه LeNet-۵ را پیاده‌سازی کنید. تمرکز اصلی بر روی نگهداری مجموع‌های جزئی درPE ها خواهد بود. در این مرحله بهتر است از یک سطح حافظه برای وزن‌ها استفاده شود. شتابدهنده ShiDianNao میتواند شما را در پیاده‌سازی خروجی‌ایستا راهنمایی کند. در پایان این مرحله شتابدهنده شما باید بتواند برای یک ورودی تصویر یک برچسب پیش‌بینی کند. دقت مدل خود را با داده‌های تست بیان کنید.

- پیاده‌سازی پشتیبانی از پراکندگی غیرساختاری

به منظور ارزیابی، طراحی پشتیبانی از پراکندگی را به گونه‌ای پیاده‌سازی کنید که قابلیت فعال یا غیرفعال شدن داشته باشد. نکاتی که در قسمت پیاده‌سازی سخت‌افزاری مربوط به پراکندگی گفته‌شد را رعایت کنید. این نکات به طور خلاصه شامل موارد زیر است:

- واحد بررسی صفر برای صرف‌نظر از ضرب‌هایی با عملوند صفر
- مسیر داده‌ای بای‌پس برای جلوگیری از اجرایMAC های غیرضروری
- مدیریت هماهنگ بین واحدهای پردازشی و سیستم حافظه برای جلوگیری از ایجاد تداخل

پس از سنتز، نتایج به‌دست‌آمده در حالت فعال و غیرفعال بودن پشتیبانی از پراکندگی از نظر موارد زیر مقایسه و تحلیل شود:

- مصرف توان
- تعداد واقعی عملیات MAC
- کارایی سیستم بر حسب کاهش سیکل‌های اجرای استنتاج یا گذردهی (توان عملیاتی)

در گزارش خود روش محاسبه و مقایسه این معیارها را نیز به‌صورت واضح شرح دهید.

- پیاده سازی با دو سطح حافظه

در این مرحله حافظه دوسطحی را پیاده‌سازی کنید. مواردی که باید مشخص شود:

- ساختار مدیریت کش و سیاست جایگزینی، اندازه خط، تکنیک پیش‌بارگذاری (در صورت استفاده از این تکنیک)
- نوع داده‌های ذخیره‌شده در هر سطح (فعال‌سازی، وزن یا مجموع‌های جزئی)
- طراحی کنترلر حافظه با استفاده از FSM یا شمارنده

پس از شبیه‌سازی، زمان محاسبات خود (تاخیر) را با حالت قبل مقایسه کنید و اثر آن بر کارایی را تحلیل کنید.

- تحلیل نتایج سنتز و مقایسه

نتایج حاصل از سنتز را گزارش کنید این نتایج را در نهایت با نتایج گزارش شده در مقاله ShiDianNao مقابسه کنید. تمرکز این مقابسه بر بروی کارایی و توان است.

برای حالت نهایی شتابدهنده (یعنی شتابدهنده با پشتیبانی از کوانتیزاسیون و پراکندگی با دو سطح حافظه) شامل موارد زیر گزارش شود.

- مصرف منابع (LUT، FF، DSP، BRAM)
- توان مصرفی
- زمان اجرای استنتاج (تاخیر)
- دقت مدل
- **مقایسه کارایی:** با استفاده از نتایج ارائه‌شده در مقاله ShiDianNao، مقدار speedup طراحی خود را نسبت به یک پردازنده CPU مرجع محاسبه و گزارش کنید. برای به دست آوردن زمان اجرای استنتاج دقیق روی CPU مدل را روی CPU ابزار google colab اجرا و نسبت به آن بسنجید.
- **مقایسه توان مصرفی:** توان مصرفی طراحی خود را با استفاده از ابزار Vivado Power Analyzer به‌دست آورده و با مقدار مصرف توان گزارش‌شده برای ShiDianNao مقایسه نمایید. سعی کنید علاوه بر مقایسه عددی، تحلیل کنید که علت اختلاف در توان مصرفی می‌تواند ناشی از چه عواملی باشد (نوع پیاده‌سازی، تکنولوژی ساخت، ابعاد آرایه، دقت محاسبات و ...).

توضیحات : خروجی‌های پروژه شامل موارد زیر است:

فایل ارسالی یک فایل Zip است. خروجی‌ها برای هر سوال، در یک زیر شاخه‌ی جدا باشد. همه زیرشاخه‌ها در فایل Zip با نام FINAL_Project_StudentNumber_Fullname.zip گنجانده شود. در صورت درخواست زمان اضافی برای انجام پروژه، ابتدا به دستیار آموزشی در تلگرام پیام دهید و پس از هماهنگی، از ۲ روز کوپن خود استفاده کنید. چنانچه سوالی دارید می‌توانید از دستیار آموزشی درس در گروه تلگرام سوال نمایید.

لطفاً حتماً از قالب آماده گزارش که قبلاً تهیه شده است، استفاده کنید. هر گونه ارسال گزارش خارج از این قالب مورد بررسی قرار نخواهد گرفت.